

Day-3-Assignment-sql

By shaaz

```
CREATE TABLE air_passenger_profile (  
    passenger_pk INT IDENTITY(1,1) PRIMARY KEY,  
    profile_id VARCHAR(10) NOT NULL UNIQUE,  
    password VARCHAR(50) NOT NULL,  
    first_name VARCHAR(50),  
    last_name VARCHAR(50),  
    address VARCHAR(100),  
    mobile_number BIGINT,  
    email_id VARCHAR(30)  
);  
GO
```

```
CREATE TABLE air_flight (  
    flight_pk INT IDENTITY(1,1) PRIMARY KEY,  
    flight_id VARCHAR(10) NOT NULL UNIQUE,  
    airline_id VARCHAR(10),  
    airline_name VARCHAR(30),  
    from_location VARCHAR(20),  
    to_location VARCHAR(20),  
    departure_time TIME,  
    arrival_time TIME,  
    duration INT,  
    total_seats INT  
);
```

```
CREATE TABLE air_flight_details (  
    flight_details_pk INT IDENTITY(1,1) PRIMARY KEY,  
    flight_id VARCHAR(10) NOT NULL,  
    flight_departure_date DATE,  
    price DECIMAL(8,2),  
    available_seats INT,  
  
    CONSTRAINT fk_flight_details_flight  
        FOREIGN KEY (flight_id)  
        REFERENCES air_flight(flight_id)
```

```
);
```

```
CREATE TABLE air_ticket_info (  
    ticket_pk INT IDENTITY(1,1) PRIMARY KEY,  
    ticket_id VARCHAR(10) NOT NULL UNIQUE,  
    profile_id VARCHAR(10) NOT NULL,  
    flight_id VARCHAR(10) NOT NULL,  
    flight_departure_date DATE,  
    status VARCHAR(10),  
  
    CONSTRAINT fk_ticket_profile  
        FOREIGN KEY (profile_id)  
        REFERENCES air_passenger_profile(profile_id),  
  
    CONSTRAINT fk_ticket_flight  
        FOREIGN KEY (flight_id)  
        REFERENCES air_flight(flight_id)  
);  
GO
```

```
CREATE TABLE air_credit_card_details (  
    card_pk INT IDENTITY(1,1) PRIMARY KEY,  
    profile_id VARCHAR(10) NOT NULL,  
    card_number BIGINT,  
    card_type VARCHAR(10),  
    expiration_month INT,  
    expiration_year INT,  
  
    CONSTRAINT fk_card_profile  
        FOREIGN KEY (profile_id)  
        REFERENCES air_passenger_profile(profile_id)  
);
```

```
SELECT * FROM air_passenger_profile;  
SELECT * FROM air_flight;  
SELECT * FROM air_flight_details;  
SELECT * FROM air_ticket_info;  
SELECT * FROM air_credit_card_details;
```

```
INSERT INTO air_passenger_profile
```

```
(profile_id, password, first_name, last_name, address, mobile_number, email_id)
VALUES
('P001', 'pwd1', 'Amit', 'Sharma', 'Chennai', 9876543210, 'amit@gmail.com'),
('P002', 'pwd2', 'Bharath', 'Simha', 'Hyderabad', 9876543220, 'bharath@gmail.com'),
('P003', 'pwd3', 'Chetan', 'Kumar', 'Chennai', 9876543230, 'chetan@gmail.com'),
('P004', 'pwd4', 'Divya', 'Rao', 'Bangalore', 9876543240, 'divya@gmail.com'),
('P005', 'pwd5', 'Esha', 'Verma', 'Chennai', 9876543250, 'esha@gmail.com');
```

```
INSERT INTO air_flight
(flight_id, airline_id, airline_name, from_location, to_location, departure_time,
arrival_time, duration, total_seats)
VALUES
('F101', 'A01', 'ABC Airlines', 'Chennai', 'Hyderabad', '08:00', '09:30', 90, 180),
('F102', 'A01', 'ABC Airlines', 'Chennai', 'Hyderabad', '14:00', '15:30', 90, 180),
('F103', 'A01', 'ABC Airlines', 'Hyderabad', 'Bangalore', '10:00', '11:15', 75, 180),
('F104', 'A01', 'ABC Airlines', 'Chennai', 'Delhi', '06:00', '08:30', 150, 200),
('F105', 'A01', 'ABC Airlines', 'Delhi', 'Hyderabad', '18:00', '20:30', 150, 200);
```

```
INSERT INTO air_flight_details
(flight_id, flight_departure_date, price, available_seats)
VALUES
('F101', '2025-04-05', 4500, 50),
('F101', '2025-05-10', 4800, 60),
('F102', '2025-04-15', 4700, 40),
('F103', '2025-04-20', 5200, 30),
('F104', '2025-06-12', 6500, 70),
('F105', '2025-04-18', 6000, 55);
```

```
INSERT INTO air_ticket_info
(ticket_id, profile_id, flight_id, flight_departure_date, status)
VALUES
('T001', 'P001', 'F101', '2025-04-05', 'BOOKED'),
('T002', 'P001', 'F102', '2025-04-15', 'BOOKED'),
('T003', 'P002', 'F101', '2025-04-05', 'BOOKED'),
('T004', 'P003', 'F101', '2025-04-05', 'BOOKED'),
('T005', 'P003', 'F103', '2025-04-20', 'BOOKED'),
('T006', 'P003', 'F102', '2025-04-15', 'BOOKED'),
('T007', 'P004', 'F104', '2025-06-12', 'BOOKED'),
('T008', 'P005', 'F101', '2025-04-05', 'BOOKED');
```

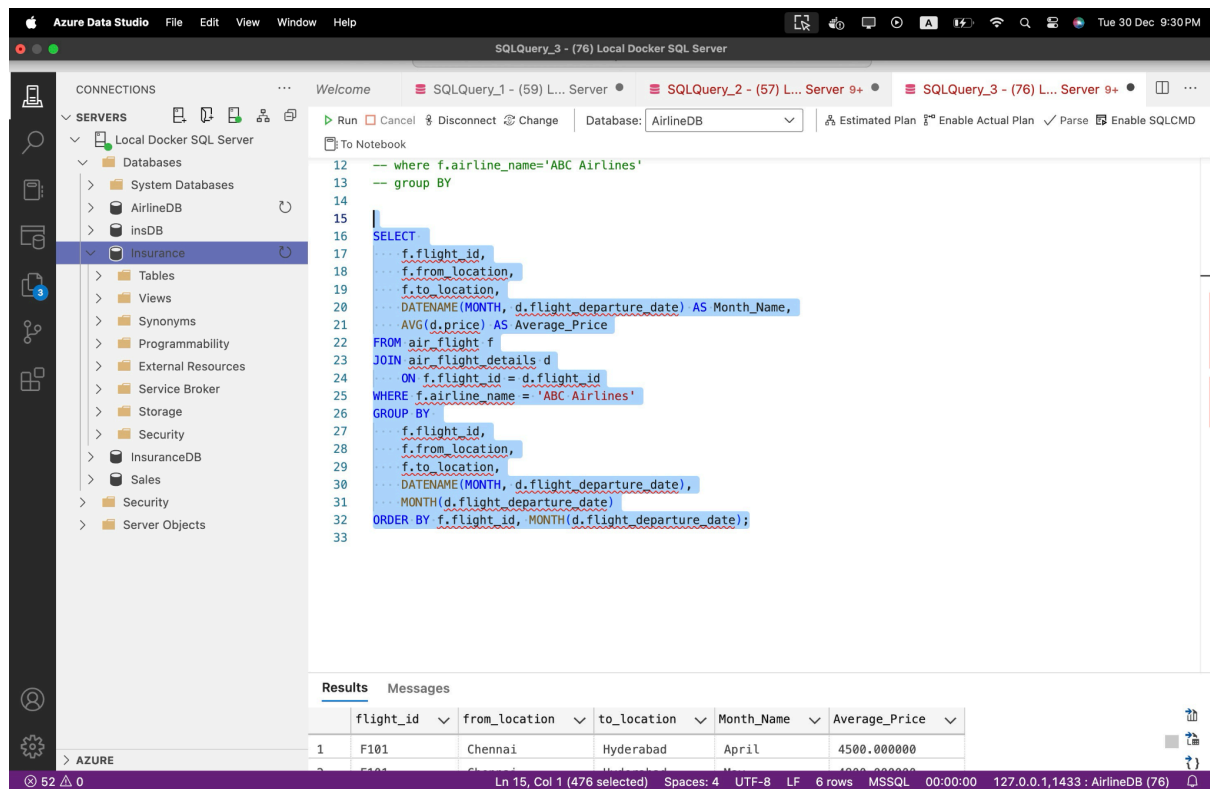
```
INSERT INTO air_credit_card_details
(profile_id, card_number, card_type, expiration_month, expiration_year)
```

```
VALUES
('P001',1111222233334444,'VISA',12,2027),
('P002',2222333344445555,'MC',11,2026),
('P003',3333444455556666,'VISA',10,2028),
('P004',4444555566667777,'AMEX',9,2026),
('P005',5555666677778888,'VISA',8,2029);
```

questions

1.

```
SELECT
    f.flight_id,
    f.from_location,
    f.to_location,
    DATENAME(MONTH, d.flight_departure_date) AS Month_Name,
    AVG(d.price) AS Average_Price
FROM air_flight f
JOIN air_flight_details d
    ON f.flight_id = d.flight_id
WHERE f.airline_name = 'ABC Airlines'
GROUP BY
    f.flight_id,
    f.from_location,
    f.to_location,
    DATENAME(MONTH, d.flight_departure_date),
    MONTH(d.flight_departure_date)
ORDER BY f.flight_id, MONTH(d.flight_departure_date);
```

2.

```

SELECT
    f.from_location,
    f.to_location,
    DATENAME(MONTH, d.flight_departure_date) AS Month_Name,
    COUNT(d.flight_departure_date) AS No_of_Services
FROM air_flight f
JOIN air_flight_details d
    ON f.flight_id = d.flight_id
GROUP BY
    f.from_location,
    f.to_location,
    DATENAME(MONTH, d.flight_departure_date),
    MONTH(d.flight_departure_date)
ORDER BY f.from_location, f.to_location, MONTH(d.flight_departure_date);

```

The screenshot shows the Azure Data Studio interface. On the left, the 'CONNECTIONS' pane shows a tree view of the 'Local Docker SQL Server' with databases like 'AirlineDB' and 'insDB'. The 'insDB' database is selected, showing its schema. The main editor displays a SQL query with line numbers 25 to 48. The query filters for 'ABC Airlines' and groups data by flight location and month. The 'Results' pane at the bottom shows a table with 5 rows of data.

SQL Query:

```

25 WHERE f.airline_name = 'ABC Airlines'
26 GROUP BY
27     f.flight_id,
28     f.from_location,
29     f.to_location,
30     DATENAME(MONTH, d.flight_departure_date),
31     MONTH(d.flight_departure_date)
32 ORDER BY f.flight_id, MONTH(d.flight_departure_date);
33
34
35 SELECT
36     f.from_location,
37     f.to_location,
38     DATENAME(MONTH, d.flight_departure_date) AS Month_Name,
39     COUNT(d.flight_departure_date) AS No_of_Services
40 FROM air_flight f
41 JOIN air_flight_details d
42     ON f.flight_id = d.flight_id
43 GROUP BY
44     f.from_location,
45     f.to_location,
46     DATENAME(MONTH, d.flight_departure_date),
47     MONTH(d.flight_departure_date)
48 ORDER BY f.from_location, f.to_location, MONTH(d.flight_departure_date);

```

Results Table:

	from_location	to_location	Month_Name	No_of_Services
1	Chennai	Delhi	June	1
2	Chennai	Hyderabad	April	2
3	Chennai	Hyderabad	May	1
4	Delhi	Hyderabad	April	1
5	Hyderabad	Bangalore	April	1

3.

SELECT

```

    f.from_location,
    f.to_location,
    DATENAME(MONTH, d.flight_departure_date) AS Month_Name,
    COUNT(d.flight_departure_date) AS No_of_Services

```

FROM air_flight f

JOIN air_flight_details d

```

    ON f.flight_id = d.flight_id

```

GROUP BY

```

    f.from_location,
    f.to_location,
    DATENAME(MONTH, d.flight_departure_date),
    MONTH(d.flight_departure_date)

```

ORDER BY f.from_location, f.to_location, MONTH(d.flight_departure_date);

The screenshot shows the Azure Data Studio interface with a SQL query executed in the AirlineDB database. The query is a SELECT statement with a JOIN and GROUP BY clause. The results are displayed in a table with 5 rows.

```

44  f.from_location,
45  f.to_location,
46  DATENAME(MONTH, d.flight_departure_date),
47  MONTH(d.flight_departure_date)
48  ORDER BY f.from_location, f.to_location, MONTH(d.flight_departure_date);
49
50
51
52  SELECT
53  f.from_location,
54  f.to_location,
55  DATENAME(MONTH, d.flight_departure_date) AS Month_Name,
56  COUNT(d.flight_departure_date) AS No_of_Services
57  FROM air_flight f
58  JOIN air_flight_details d
59  ON f.flight_id = d.flight_id
60  GROUP BY
61  f.from_location,
62  f.to_location,
63  DATENAME(MONTH, d.flight_departure_date),
64  MONTH(d.flight_departure_date)
65  ORDER BY f.from_location, f.to_location, MONTH(d.flight_departure_date);

```

	from_location	to_location	Month_Name	No_of_Services
1	Chennai	Delhi	June	1
2	Chennai	Hyderabad	April	2
3	Chennai	Hyderabad	May	1
4	Delhi	Hyderabad	April	1
5	Hyderabad	Bangalore	April	1

4.

```

SELECT
    p.profile_id,
    p.first_name,
    p.address,
    COUNT(t.ticket_id) AS No_of_Tickets
FROM air_passenger_profile p
JOIN air_ticket_info t
    ON p.profile_id = t.profile_id
GROUP BY p.profile_id, p.first_name, p.address
HAVING COUNT(t.ticket_id) = (
    SELECT MAX(cnt)
    FROM (
        SELECT COUNT(ticket_id) cnt
        FROM air_ticket_info
        GROUP BY profile_id
    )
)

```

```
) x
)
ORDER BY p.first_name;
```

The screenshot shows the Azure Data Studio interface. On the left, the 'CONNECTIONS' pane shows a tree view of the 'Local Docker SQL Server' with databases like 'AirlineDB' and 'insDB'. The 'insDB' database is selected. The main editor displays a SQL query (lines 67-88) that selects passenger information and ticket counts, grouped by passenger profile and ordered by first name. The 'Results' pane at the bottom shows a single row of data for profile 'P003' (Chetan, Chennai) with 3 tickets.

```

67
68
69 SELECT
70     p.profile_id,
71     p.first_name,
72     p.address,
73     COUNT(t.ticket_id) AS No_of_Tickets
74 FROM air_passenger_profile p
75 JOIN air_ticket_info t
76     ON p.profile_id = t.profile_id
77 GROUP BY p.profile_id, p.first_name, p.address
78 HAVING COUNT(t.ticket_id) = (
79     SELECT MAX(cnt)
80     FROM (
81         SELECT COUNT(ticket_id) cnt
82         FROM air_ticket_info
83         GROUP BY profile_id
84     ) x
85 )
86 ORDER BY p.first_name;
87
88

```

	profile_id	first_name	address	No_of_Tickets
1	P003	Chetan	Chennai	3

5.

```

SELECT
    p.profile_id,
    p.first_name,
    p.last_name,
    t.flight_id,
    t.flight_departure_date,
    COUNT(t.ticket_id) AS No_of_Tickets
FROM air_ticket_info t
JOIN air_passenger_profile p ON t.profile_id = p.profile_id
JOIN air_flight f ON t.flight_id = f.flight_id
WHERE f.from_location = 'Chennai'
    AND f.to_location = 'Hyderabad'
GROUP BY
    p.profile_id, p.first_name, p.last_name,
    t.flight_id, t.flight_departure_date
ORDER BY p.profile_id, t.flight_id, t.flight_departure_date;
```

The screenshot shows the Azure Data Studio interface with a SQL query executed against the 'AirlineDB' database. The query is a complex JOIN statement involving 'air_passenger_profile', 'air_ticket_info', and 'air_flight' tables. The results are displayed in a table with 7 columns: profile_id, first_name, last_name, flight_id, flight_departure_date, and No_of_Tickets. The results show 6 rows of data.

```

86 ORDER BY p.first_name;
87
88
89
90 SELECT
91     p.profile_id,
92     p.first_name,
93     p.last_name,
94     t.flight_id,
95     t.flight_departure_date,
96     COUNT(t.ticket_id) AS No_of_Tickets
97 FROM air_ticket_info t
98 JOIN air_passenger_profile p ON t.profile_id = p.profile_id
99 JOIN air_flight f ON t.flight_id = f.flight_id
100 WHERE f.from_location = 'Chennai'
101 AND f.to_location = 'Hyderabad'
102 GROUP BY
103     p.profile_id, p.first_name, p.last_name,
104     t.flight_id, t.flight_departure_date
105 ORDER BY p.profile_id, t.flight_id, t.flight_departure_date;
106
107

```

	profile_id	first_name	last_name	flight_id	flight_departure_date	No_of_Tickets
1	P001	Amit	Sharma	F101	2025-04-05	1
2	P001	Amit	Sharma	F102	2025-04-15	1
3	P002	Bharath	Simha	F101	2025-04-05	1
4	P003	Chetan	Kumar	F101	2025-04-05	1
5	P003	Chetan	Kumar	F102	2025-04-15	1
6	P005	Esha	Verma	F101	2025-04-05	1

Ln 90, Col 1 (501 selected) Spaces: 4 UTF-8 LF 6 rows MSSQL 00:00:00 127.0.0.1,1433 : AirlineDB (76)

6.

SELECT

f.flight_id,
f.from_location,
f.to_location,
d.price

FROM air_flight f

JOIN air_flight_details d

ON f.flight_id = d.flight_id

WHERE MONTH(d.flight_departure_date) = 4;

The screenshot shows the Azure Data Studio interface. On the left, the 'SERVERS' pane shows a 'Local Docker SQL Server' with a database named 'AirlineDB'. The 'insDB' database is selected, showing a list of tables including 'air_flight' and 'air_flight_details'. The main editor displays a SQL query with line numbers 100 to 119. The query filters for flights from Chennai to Hyderabad in April, grouped by flight details and ordered by profile ID. The 'Results' pane at the bottom shows a table with 4 rows of flight data.

```

100 WHERE f.from_location = 'Chennai'
101 AND f.to_location = 'Hyderabad'
102 GROUP BY
103     p.profile_id, p.first_name, p.last_name,
104     t.flight_id, t.flight_departure_date
105 ORDER BY p.profile_id, t.flight_id, t.flight_departure_date;
106
107
108
109
110 SELECT
111     f.flight_id,
112     f.from_location,
113     f.to_location,
114     d.price
115 FROM air_flight f
116 JOIN air_flight_details d
117     ON f.flight_id = d.flight_id
118 WHERE MONTH(d.flight_departure_date) = 4;
119

```

	flight_id	from_location	to_location	price
1	F101	Chennai	Hyderabad	4500.00
2	F102	Chennai	Hyderabad	4700.00
3	F103	Hyderabad	Bangalore	5200.00
4	F105	Delhi	Hyderabad	6000.00

7.

```

SELECT
    f.flight_id,
    f.from_location,
    f.to_location,
    AVG(d.price) AS Price
FROM air_flight f
JOIN air_flight_details d
    ON f.flight_id = d.flight_id
GROUP BY f.flight_id, f.from_location, f.to_location
ORDER BY f.flight_id, f.from_location, f.to_location;

```

The screenshot shows the Azure Data Studio interface. On the left, the 'SERVERS' pane shows a 'Local Docker SQL Server' with a database named 'AirlineDB'. The main editor displays a SQL query (lines 111-132) that joins 'air_flight' and 'air_flight_details' tables, filters for flights departing in the 4th month, and calculates the average price. The 'Results' pane shows the output of the query, which is a table with 5 rows and 4 columns: flight_id, from_location, to_location, and Price.

```

111 f.flight_id,
112 f.from_location,
113 f.to_location,
114 d.price
115 FROM air_flight f
116 JOIN air_flight_details d
117 ON f.flight_id = d.flight_id
118 WHERE MONTH(d.flight_departure_date) = 4;
119
120
121 SELECT
122 f.flight_id,
123 f.from_location,
124 f.to_location,
125 AVG(d.price) AS Price
126 FROM air_flight f
127 JOIN air_flight_details d
128 ON f.flight_id = d.flight_id
129 GROUP BY f.flight_id, f.from_location, f.to_location
130 ORDER BY f.flight_id, f.from_location, f.to_location;
131
132

```

	flight_id	from_location	to_location	Price
1	F101	Chennai	Hyderabad	4650.000000
2	F102	Chennai	Hyderabad	4700.000000
3	F103	Hyderabad	Bangalore	5200.000000
4	F104	Chennai	Delhi	6500.000000
5	F105	Delhi	Hyderabad	6000.000000

8.

SELECT DISTINCT

p.profile_id,

CONCAT(p.first_name, ' ', p.last_name) AS customer_name,

p.address

FROM air_passenger_profile p

JOIN air_ticket_info t ON p.profile_id = t.profile_id

JOIN air_flight f ON t.flight_id = f.flight_id

WHERE f.from_location = 'Chennai'

AND f.to_location = 'Hyderabad'

ORDER BY p.profile_id;

Azure Data Studio interface showing a SQL query and its results.

SQL Query:

```

122     f.flight_id,
123     f.from_location,
124     f.to_location,
125     AVG(d.price) AS Price
126 FROM air_flight f
127 JOIN air_flight_details d
128     ON f.flight_id = d.flight_id
129 GROUP BY f.flight_id, f.from_location, f.to_location
130 ORDER BY f.flight_id, f.from_location, f.to_location;
131
132 SELECT DISTINCT
133     p.profile_id,
134     CONCAT(p.first_name, ' ', p.last_name) AS customer_name,
135     p.address
136 FROM air_passenger_profile p
137 JOIN air_ticket_info t ON p.profile_id = t.profile_id
138 JOIN air_flight f ON t.flight_id = f.flight_id
139 WHERE f.from_location = 'Chennai'
140     AND f.to_location = 'Hyderabad'
141 ORDER BY p.profile_id;
142
143

```

Results:

	profile_id	customer_name	address
1	P001	Amit, Sharma	Chennai
2	P002	Bharath, Simha	Hyderabad
3	P003	Chetan, Kumar	Chennai
4	P005	Esha, Verma	Chennai

9.

```

SELECT profile_id
FROM air_ticket_info
GROUP BY profile_id
HAVING COUNT(ticket_id) = (
    SELECT MAX(cnt)
    FROM (
        SELECT COUNT(ticket_id) cnt
        FROM air_ticket_info

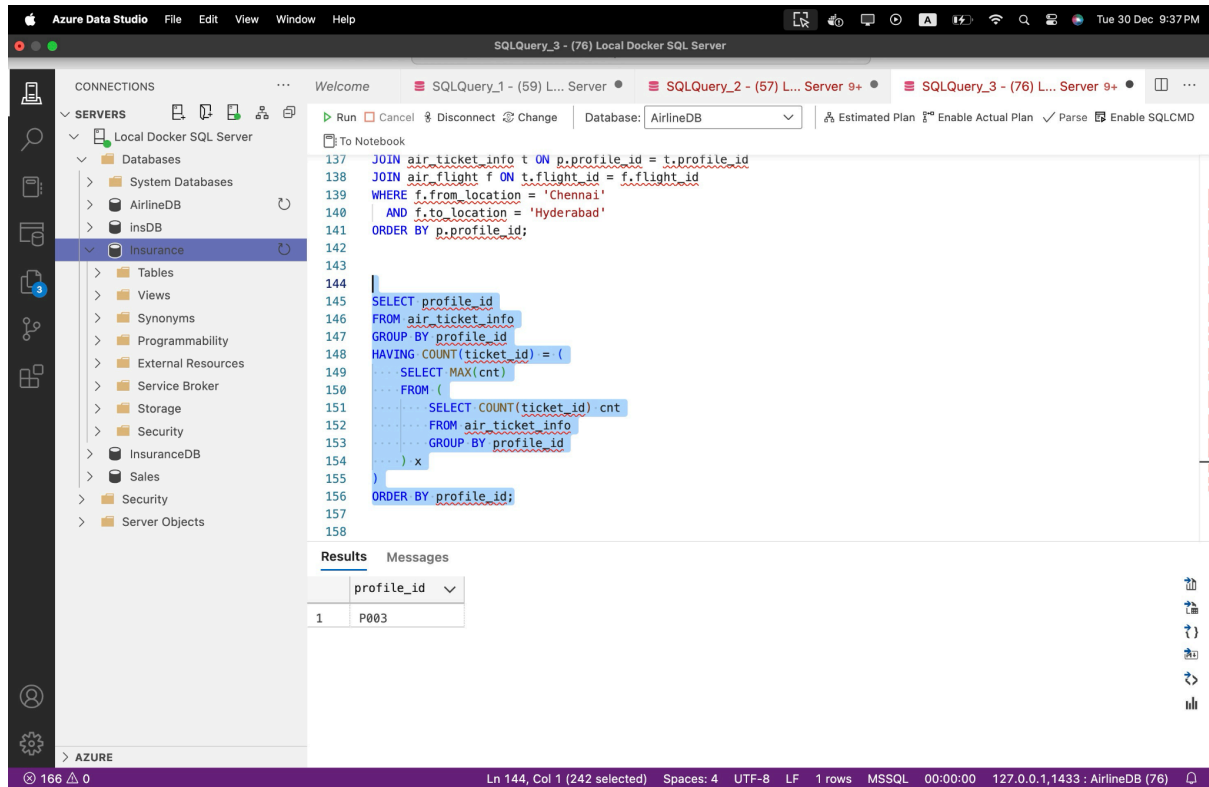
```



```

        GROUP BY profile_id
    ) x
)
ORDER BY profile_id;

```



10.

```

SELECT
    f.flight_id,
    f.from_location,
    f.to_location,
    COUNT(t.ticket_id) AS No_of_Tickets
FROM air_flight f
JOIN air_ticket_info t
    ON f.flight_id = t.flight_id
GROUP BY f.flight_id, f.from_location, f.to_location
HAVING COUNT(t.ticket_id) >= 1
ORDER BY f.flight_id;

```

179 0

Ln 162, Col 1 (284 selected) Spaces: 4 UTF-8 LF 4 rows MSSQL 00:00:00 127.0.0.1,1433 : AirlineDB (76)

153 GROUP BY profile_id
154) x
155)
156 ORDER BY profile_id;
157
158
159
160
161
162 SELECT
163 f.flight_id,
164 f.from_location,
165 f.to_location,
166 COUNT(t.ticket_id) AS No_of_Tickets
167 FROM air_flight f
168 JOIN air_ticket_info t
169 ON f.flight_id = t.flight_id
170 GROUP BY f.flight_id, f.from_location, f.to_location
171 HAVING COUNT(t.ticket_id) >= 1
172 ORDER BY f.flight_id;
173
174

Results Messages

	flight_id	from_location	to_location	No_of_Tickets
1	F101	Chennai	Hyderabad	4
2	F102	Chennai	Hyderabad	2
3	F103	Hyderabad	Bangalore	1
4	F104	Chennai	Delhi	1

CONNECTIONS

SERVERS

Local Docker SQL Server

Databases

System Databases

AirlineDB

insDB

Insurance

Tables

Views

Synonyms

Programmability

External Resources

Service Broker

Storage

Security

InsuranceDB

Sales

Security

Server Objects

SQLQuery_3 - (76) Local Docker SQL Server

Database: AirlineDB

Estimated Plan Enable Actual Plan Parse Enable SQLCMD

179 0

Ln 162, Col 1 (284 selected) Spaces: 4 UTF-8 LF 4 rows MSSQL 00:00:00 127.0.0.1,1433 : AirlineDB (76)